In [57]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
%matplotlib inline
import matplotlib
matplotlib.rcParams['figure.figsize'] = (20,20)
from scipy.stats import skew,kurtosis
```

DATA CLEANING AND PREPROCESSING

In [14]:
```python
# LOAD THE DATASET
data = pd.read_csv('/Users/punyashree/Documents/MY_SECOND_PROJECT/Bengaluru_House_Data.csv')
```

In [18]:
```python
data.head()
```

Out[18]:

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056	2.0	1.0	39.07
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00
2	Built-up Area	Ready To Move	Uttarahalli	3 BHK	NaN	1440	2.0	3.0	62.00
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Soiewre	1521	3.0	1.0	95.00
4	Super built-up Area	Ready To Move	Kothanur	2 BHK	NaN	1200	2.0	1.0	51.00

In [20]:
```python
data.size
```

Out[20]:
```
119880
```

In [24]:
```python
data.shape
```

Out[24]:
```
(13320, 9)
```

In [28]:
```python
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13320 entries, 0 to 13319
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   area_type     13320 non-null  object 
 1   availability  13320 non-null  object 
 2   location      13319 non-null  object 
 3   size          13304 non-null  object 
 4   society       7818 non-null   object 
 5   total_sqft    13320 non-null  object 
 6   bath          13247 non-null  float64
 7   balcony       12711 non-null  float64
 8   price         13320 non-null  float64
dtypes: float64(3), object(6)
memory usage: 936.7+ KB
```

In [38]:
```python
data.ndim
```

Out[38]:
```
2
```

In [30]:
```python
data.describe()
```

Out[30]:

	bath	balcony	price
count	13247.000000	12711.000000	13320.000000
mean	2.692610	1.584376	112.565627
std	1.341458	0.817263	148.971674
min	1.000000	0.000000	8.000000
25%	2.000000	1.000000	50.000000
50%	2.000000	2.000000	72.000000
75%	3.000000	2.000000	120.000000
max	40.000000	3.000000	3600.000000

In [51]:
```python
# WE CAN CHECK BY AGGREGATION TECHNIQUES THAT HOW MANY DISTINCT LOCATIONS AND AREA_TYPE ARE THERE IN GIVEN DATASET

data.groupby('area_type')['area_type'].agg('count')
```

Out[51]:
```
area_type
Built-up  Area          2418
Carpet  Area              87
Plot  Area              2025
Super built-up  Area    8790
Name: area_type, dtype: int64
```

In [53]:
```python
data.groupby('location')['location'].agg('count')
```

Out[53]:
```
location
 Anekal                 1
 Banaswadi              1
 Basavangudi            1
 Bhoganhalli            1
 Devarabeesana Halli    6
                       ..
t.c palya               1
tc.palya                4
vinayakanagar           1
white field,kadugodi    1
whitefiled              1
Name: location, Length: 1305, dtype: int64
```

In [55]:
```python
data.groupby('size')['size'].agg('count')
```

Out[55]:
```
size
1 BHK          538
1 Bedroom      105
1 RK            13
10 BHK           2
10 Bedroom      12
11 BHK           2
11 Bedroom       2
12 Bedroom       1
13 BHK           1
14 BHK           1
16 BHK           1
18 Bedroom       1
19 BHK           1
2 BHK         5199
2 Bedroom      329
27 BHK           1
3 BHK         4310
3 Bedroom      547
4 BHK          591
4 Bedroom      826
43 Bedroom       1
5 BHK           59
5 Bedroom      297
6 BHK           30
6 Bedroom      191
7 BHK           17
7 Bedroom       83
8 BHK            5
8 Bedroom       84
9 BHK            8
9 Bedroom       46
Name: size, dtype: int64
```

In [65]:
```python
# DATA CLEANING STARTS WITH REMOVING THE NULL VALUES FROM DATASET

data.isnull().sum()
```

Out[65]:
```
area_type          0
availability       0
location           1
size              16
society         5502
total_sqft         0
bath              73
balcony          609
price              0
dtype: int64
```

In [71]:
```python
# HOW TO DECIDE TO DROP THE NULL VALUE COLUMNS OR NOT ???????????
# LOOK INTO THE DATSET ROWS IF WE HAVE MORE ROWS DROPPING A CERTAIN ROWS WON'T AFFECT THE MODEL TRAINING
# IN SUCH CASES WE DROP ALL THE NULL VALUES FROM DATASET
# IF WE DONT WAN'T TO DROP [SMALL DATASET] --> TAKE MEDIAN OF ENTIRE COLUMN AND FILLNA() WITH MEDIAN VALUES


data_ = data.dropna()
```

In [75]:
```python
data_.isnull().sum()

# WE CAN SEE THE NEW DATA IN A DATA FRAME WITH ZERO NULL VALUES AFTER DROPPING
```

Out[75]:
```
area_type       0
availability    0
location        0
size            0
society         0
total_sqft      0
bath            0
balcony         0
price           0
dtype: int64
```

In [119]:
```python
# WE CAN CALCULATE SKEWNESS TO CHECK THE DISTRIBUTION OF DATA AROUND MEAN TO MEASURE TAILNESS
skew_bath = stats.skew(data_['bath'])
print(skew_bath)

skew_price = stats.skew(data_['price'])
print(skew_price)

# POSITIVE SKEW/RIGHT SKEW --. MOST OF THE DATA LIES TOWARDS LEFT SIDE OF THE MEAN OF COLUMN[TAIL RIGHT]
```

```
1.5104764188098923
7.713187931893433
```

In [157]:
```python
# KURTOSIS
kurtosis_bath = stats.kurtosis(data_["bath"])
print(kurtosis_bath)

kurtosis_price = stats.kurtosis(data_['price'])
print(kurtosis_price)

kurtosis_balcony = stats.kurtosis(data_['balcony'])
print(kurtosis_balcony)
# HERE WE CAN CONCLUDE IF THE KV>3 DATA IS LEPTOKURTIC AND PEAK TAILS --> MORE NUMBER OF OUTLIERS
# DATA SPREAD OUT IN NUMBERS HAS MANY DIFFERENT HIGH AND LOW VALUES.
```

```
3.9176288044574425
116.88932264846615
-0.5122764297630176
```

In [163]:
```python
# AS IN DATA WE CAN SEE THE SIZE HAS TWO DIFFERENT NAMES [BEDROOM AND BHK]
# AS WE KNOW BOTH ARE SAME WE CAN REMOVE THOSE TO GET MORE ACCURACY
# TO CHECK FOR THE PARTICULAR COLUMN WE CAN USE UNIQUE FUNCTION

data_['size'].unique()
```

Out[163]:
```
array(['2 BHK', '4 Bedroom', '3 BHK', '3 Bedroom', '1 RK', '4 BHK',
       '1 BHK', '5 BHK', '11 BHK', '5 Bedroom', '9 BHK', '2 Bedroom',
       '6 BHK', '7 BHK', '6 Bedroom'], dtype=object)
```

In [165]:
```python
data_['bedrooms'] = data_['size'].apply(lambda x : int(x.split(" ")[0]))
```

```
/var/folders/_x/72pzxcts2k7bb7ows40jb51r0000gn/T/ipykernel_52220/3827235998.py:1: SettingWithCopyWarning: 
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data_["bedrooms"] = data_['size'].apply(lambda x : int(x.split(" ")[0]))
```

In [169]:
```python
# AS WE HAVE APPLIED THE FUNCTION WE CAN NOW SE THE NEW COLUMN bedrooms IS ADDED HERE

data_.head()
```

Out[169]:

	area_type	availability	location	size	society	total_sqft	bath	balcony	price	bedrooms
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056	2.0	1.0	39.07	2
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00	4
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Soiewre	1521	3.0	1.0	95.00	3
5	Super built-up Area	Ready To Move	Whitefield	2 BHK	DuenaTa	1170	2.0	1.0	38.00	2
11	Plot Area	Ready To Move	Whitefield	4 Bedroom	Prrry M	2785	5.0	3.0	295.00	4

In [195]:
```python
## WE CAN SIMPLY PERFORM SOME ACTIONS TO CHECKOUT THE DATA
data_[data_.bedrooms>20]
```

Out[195]:

area_type	availability	location	size	society	total_sqft	bath	balcony	price	bedrooms

In [197]:
```python
data_.total_sqft.unique()
```

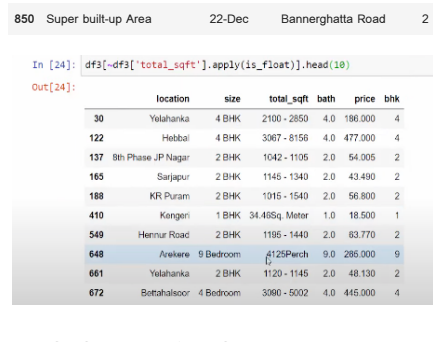
Out[197]:
```
array(['1056', '2600', '1521', ..., '1919', '2111', '4689'], dtype=object)
```

In [204]:
```python
# IF WE SEE ANY RANGE OF VALUES WE CAN CONVERT TO FLOAT AND TO CHECK WHAT KIND OF VARIATIONS WE HAVE
# WE HAVE TO TAKE AVERAGE OF RANGE OF VALUES -> TO CONVERT TO SINGLE NUMBER
# FIRST WE HAVE TO CHECK IS THE GIVEN VALUE IS FLOAT OR NOT

# WE TRY TO CONVERT THE VALUES TO FLOAT IF IT IS NOT FLOAT[LIKE RANGE] IT GIVES EXCEPT CLAUSE


def is_float(x):
    try:
        float(x)
    except:
        return False
    return True
```

In [214]:
```python
# NOW IF WE APPLY IS_FLOAT FUNCTION FUNCTION IT APPLIES
# WE WANT ONLY THE VALUES WHICH ARE NOT FLOAT
# BY USING NEGATION WE CAN SORT EASILY
# IF WE CAN ALSO SEE CASES LIKE STR IN DATA

data_[~data_['total_sqft'].apply(is_float)].head(10)
```

Out[214]:

	area_type	availability	location	size	society	total_sqft	bath	balcony	price	bedrooms
30	Super built-up Area	19-Dec	Yelahanka	4 BHK	LedorSa	2100 - 2850	4.0	0.0	186.000	4
122	Super built-up Area	18-Mar	Hebbal	4 BHK	SNontle	3067 - 8156	4.0	0.0	477.000	4
137	Super built-up Area	19-Mar	8th Phase JP Nagar	2 BHK	Vaarech	1042 - 1105	2.0	0.0	54.005	2
165	Super built-up Area	18-Dec	Sarjapur	2 BHK	Kinuerg	1145 - 1340	2.0	0.0	43.490	2
188	Super built-up Area	Ready To Move	KR Puram	2 BHK	MCvarar	1015 - 1540	2.0	0.0	56.800	2
549	Super built-up Area	18-Sep	Hennur Road	2 BHK	Shxorm	1195 - 1440	2.0	0.0	63.770	2
661	Super built-up Area	Ready To Move	Yelahanka	2 BHK	Rarthne	1120 - 1145	2.0	0.0	48.130	2
672	Built-up Area	18-Mar	Bettahalsoor	4 Bedroom	Toainnt	3090 - 5002	4.0	0.0	445.000	4
772	Super built-up Area	20-Dec	Banashankari Stage VI	2 BHK	Brens7	1160 - 1195	2.0	0.0	59.935	2
850	Super built-up Area	22-Dec	Bannerghatta Road	2 BHK	PrarePa	1115 - 1130	2.0	0.0	58.935	2



In [24]: `df3[~df3['total_sqft'].apply(is_float)].head(10)`

Out[24]:

	location	size	total_sqft	bath	price	bhk
30	Yelahanka	4 BHK	2100 - 2850	4.0	186.000	4
122	Hebbal	4 BHK	3067 - 8156	4.0	477.000	4
137	8th Phase JP Nagar	2 BHK	1042 - 1105	2.0	54.005	2
165	Sarjapur	2 BHK	1145 - 1340	2.0	43.490	2
188	KR Puram	2 BHK	1015 - 1540	2.0	56.800	2
410	Kengeri	1 BHK	34.46Sq. Meter	1.0	18.500	1
549	Hennur Road	2 BHK	1195 - 1440	2.0	63.770	2
648	Arekere	9 Bedroom	4125Perch	9.0	265.000	9
661	Yelahanka	2 BHK	1120 - 1145	2.0	48.130	2
672	Bettahalsoor	4 Bedroom	3090 - 5002	4.0	445.000	4

THIS IS THE MAIN STEP

- IF INCOMING DATA IS UNSTRUCTURED
- OUTLIERS
- NULL_VALUES
- MAJORLY_WE SPEND TIME ON DATA CLEANING

In [233]:
```python
# SO NOW HERE IF WE HAVE RANGE WE TAKE --> AVERAGE OF THOSE
# IF WE HAVE VALUES WITH STRING WE CAN IGNORE THOSE VALUES
# IF WE NEED A STILL SOPISTICATED MANNER WE CAN DO UNIT CONVERSION

def convert_sqft_to_num(x):
    tokens = x.split("-")
    if len(tokens) == 2:
        return (float(tokens[0])+float(tokens[1]))/2
    try:
        return float(x)
    except:
        return None
```

In [237]:
```python
# CHECK THE FUNCTION
convert_sqft_to_num('21')
```

Out[237]:
```
21.0
```

In [239]:
```python
convert_sqft_to_num('2133 - 3222')
```

Out[239]:
```
2677.5
```

In [241]:
```python
# REMEMBER WE ARE CREATING A NEW DATAFRAME IN EACH STAGE IN OUR DATA PROCESSING TECHNIQUE
data_cleaning = data_.copy()
data_cleaning['total_sqft'] = data_cleaning['total_sqft'].apply(convert_sqft_to_num)
data_cleaning.head()
```

Out[241]:

	area_type	availability	location	size	society	total_sqft	bath	balcony	price	bedrooms
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056.0	2.0	1.0	39.07	2
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600.0	5.0	3.0	120.00	4
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Soiewre	1521.0	3.0	1.0	95.00	3
5	Super built-up Area	Ready To Move	Whitefield	2 BHK	DuenaTa	1170.0	2.0	1.0	38.00	2
11	Plot Area	Ready To Move	Whitefield	4 Bedroom	Prrry M	2785.0	5.0	3.0	295.00	4

In [263]:
```python
# OVERALL REMOVE UNWANTED COLUMNS DIRECTLY WHICH WE CAN SEE ,THAT DOESN'T ADD MUCH VALUE TO PRICE PREDICTION

data_cleaned = data_cleaning.drop(['area_type','society','balcony','availability'],axis = 'columns')

data_cleaned.head()
```

Out[263]:

	location	size	total_sqft	bath	price	bedrooms
0	Electronic City Phase II	2 BHK	1056.0	2.0	39.07	2
1	Chikka Tirupathi	4 Bedroom	2600.0	5.0	120.00	4
3	Lingadheeranahalli	3 BHK	1521.0	3.0	95.00	3
5	Whitefield	2 BHK	1170.0	2.0	38.00	2
11	Whitefield	4 Bedroom	2785.0	5.0	295.00	4

In [279]:
```python
data_cleaned[['bath','price']].corr()
```

Out[279]:

	bath	price
bath	1.000000	0.594844
price	0.594844	1.000000

In [281]:
```python
data_cleaned[['bedrooms','price']].corr()
```

Out[281]:

	bedrooms	price
bedrooms	1.000000	0.526322
price	0.526322	1.000000

In [293]:
```python
# CORELATION [IF DATA IS MONOTONIC]
```

In [287]:
```python
data_cleaned[['total_sqft','price']].corr(method = 'spearman')
```

Out[287]:

	total_sqft	price
total_sqft	1.000000	0.835705
price	0.835705	1.000000

In [291]:
```python
data_cleaned[['bedrooms','price']].corr(method= 'spearman')
```

Out[291]:

	bedrooms	price
bedrooms	1.000000	0.669053
price	0.669053	1.000000

In [295]:
```python
data_cleaned[['bath','price']].corr(method = 'spearman')
```

Out[295]:

	bath	price
bath	1.000000	0.718666
price	0.718666	1.000000

FEATURE ENGINEERING

In [309]:
```python
# AS WE ALL KNOW IN REAL ESTATE PREDICTION PRICE PER SQFT IS VERY MUCH USEFUL
# AS WE HAVE TO LOOK INTO DATA AND ACT ACCORDINGLY LET'S CRREATE A COLUMN CALLED PRICE PER SQFT
# AS PRICE IS  IN LAKS IN THE DATA


data_clean = data_cleaned.copy()
data_clean['price_per_sqft'] = data_clean['price'] * 100000/data_clean['total_sqft']
```

In [311]:
```python
data_clean.head()
```

Out[311]:

	location	size	total_sqft	bath	price	bedrooms	price_per_sqft
0	Electronic City Phase II	2 BHK	1056.0	2.0	39.07	2	3699.810606
1	Chikka Tirupathi	4 Bedroom	2600.0	5.0	120.00	4	4615.384615
3	Lingadheeranahalli	3 BHK	1521.0	3.0	95.00	3	6245.890861
5	Whitefield	2 BHK	1170.0	2.0	38.00	2	3247.863248
11	Whitefield	4 Bedroom	2785.0	5.0	295.00	4	10592.459605

In [316]:
```python
# LET'S ALSO CHECK FOR LOCATION COLUMN IF THERE ARE TOO MANY LOCATIONS MAY ARISE PROBLEM
# WE'LL HAVE TO CHECK HOW MANY ROWS ARE THERE PER LOCATION
# CHECK HOW MANY LOCATIONS ARE THERE

data_clean.location.unique()
```

Out[316]:
```
array(['Electronic City Phase II', 'Chikka Tirupathi',
       'Lingadheeranahalli', 'Whitefield', '7th Phase JP Nagar',
       'Sarjapur', 'Mysore Road', 'Bisuvanahalli',
       'Raja Rajeshwari Nagar', 'Ramakrishnappa Layout', 'Binny Pete',
       'Thanisandra', ' Thanisandra', 'Electronic City',
       'Ramagondanahalli', 'Yelahanka', 'Hebbal', 'Kanakpura Road',
       'Kundalahalli', 'Sarjapur  Road', 'Ganga Nagar', 'Doddathoguru',
       'Adarsh Nagar', 'Bhoganhalli', 'Lakshminarayana Pura',
       'Begur Road', 'Varthur', 'Gunjur', 'Hegde Nagar', 'Haralur Road',
       'Hennur Road', 'Cholanayakanahalli', 'Kodanda Reddy Layout',
       'EPIP Zone', 'Dasanapura', 'Kasavanhalli', 'Sanjay nagar',
       'Kengeri', 'Yeshwanthpur', 'Chandapura', 'Kothanur',
       'Green View Layout', 'Shantiniketan Layout', 'Rajaji Nagar',
       'Devanahalli', 'Byatarayanapura', 'Akshaya Nagar',
       'LB Shastri Nagar', 'Hormavu', 'Peenya', 'Kudlu Gate',
       '8th Phase JP Nagar', 'Chandra Layout', 'Anandapura',
       'Kengeri Satellite Town', 'Basavanapura', 'Kannamangala',
       'Hulimavu', 'Hosa Road', 'Keshava Nagar', 'RMV Extension',
       'Tejaswini Nagar', 'Jai Bheema Nagar', 'Attibele',
       'CV Raman Nagar', 'Malleshwaram', 'Hebbal Kempapura',
       'Vijayanagar', 'KR Puram', 'Marathahalli', 'Pattandur Agrahara',
       'HSR Layout', 'Kadugodi', 'Kogilu', 'Panathur', 'Kammasandra',
       'Electronics City Phase 1', 'Tala Cauvery Layout', 'Dasarahalli',
       'Koramangala', 'Muthurayya Swamy Layout', 'Budigere',
       'Dodda Nekkundi Extension', 'Mylasandra', 'Kalyan nagar',
       'Ashwath Nagar', 'Ncpr Industrial Layout', 'Meenakunte',
       'OMBR Layout', 'Coffee Board Layout', 'Ambedkar Nagar',
       'Geleyara Balaga Layout', 'Kalena Agrahara', 'Talaghattapura',
       'Balagere', 'Jigani', 'Gollarapalya Hosahalli', 'Old Madras Road',
       'Silver Springs Layout', '9th Phase JP Nagar', 'Jakkur',
       'Maruthi Sevanagar', 'RMV 2nd Stage', 'Singasandra', 'AECS Layout',
       'Mallasandra', 'Begur', 'JP Nagar', 'Sunder Ram Shetty Nagar',
       'Motappa Layout', 'Kaval Byrasandra', 'Kaggalipura',
       'Basavanna Nagar', '6th Phase JP Nagar', 'Ulsoor', 'Uttarahalli',
       'Thigalarapalya', ' Devarachikkanahalli', 'Bommasandra',
       'Prashanth Nagar', 'Suragajakkanahalli', 'Ardendale', 'Harlur',
       'Sampigehalli', 'Kodihalli', 'Magadi Road', 'Narayanapura',
       'Hennur', '5th Phase JP Nagar', 'Kodigehaali', 'Bannerghatta Road',
       'Gopalapura', 'Billekahalli', 'Jalahalli', 'Sompura',
       'Ashirvad Colony', 'Dodda Nekkundi', 'Hosur Road', 'Amco Colony',
       'Ambalipura', 'Hoodi', 'Samethanahalli', 'Brookefield',
       'Suddaguntepalya', 'Udayapur Village', 'Bellandur', 'Vittasandra',
       'Giri Nagar', 'Chikkabidarakallu', '1 Giri Nagar', 'Hoysalanagar',
       'Defence Colony', 'Amruthahalli', 'Patelappa Layout',
       'Subramanyapura', '3rd Block Hrbr Layout', 'Surabhi Layout',
       'Omkar Nagar', 'Kambipura', 'VHBCS Layout', 'Rajiv Nagar',
       'Gattahalli', 'Arekere', 'Mico Layout', 'Munnekollal',
       'Banashankari Stage III', 'Doorvani Nagar', 'JCR Layout',
       'Nehru Nagar', 'Sneha Colony', 'Konanakunte', 'Ashwini layout',
       'Gottigere', 'HRBR Layout', 'Kanakapura', 'Tumkur Road',
       'Hosahalli', 'Jalahalli West', 'GM Palaya', 'Jalahalli East',
       'Hosakerehalli', 'Nagondanahalli', 'Shanthala Nagar',
       'Bettahalsoor', 'Ambedkar Colony', 'Avalahalli', 'Prakruthi Nagar',
       'Abbigere', 'Tindlu', 'Green Garden Layout', 'Gubbalala',
       'Dairy Circle', 'Narayana Nagar 1st Block', 'KSRTC Layout',
       'New Gurappana Palya', 'Palanahalli', 'Vadarpalya', 'Kudlu',
       'Old Airport Road', 'Vishwapriya Layout', 'Banashankari Stage VI',
       'Battarahalli', 'HMT Layout', 'Kaggadasapura', 'ITI Layout',
       'Yelahanka New Town', 'Sahakara Nagar', 'Rachenahalli',
       'Kodbisanhalli', 'Kodichikkanahalli', 'Bendiganahalli',
       'Ferrar Nagar', 'Green Glen Layout', 'M.G Road',
       'Horamavu Banaswadi', '1st Phase JP Nagar', 'Kaverappa Layout',
       'Devarabisanahalli', 'Somasundara Palya', 'Vidyaranyapura',
       'Babusapalaya', 'Nagappa Reddy Layout', 'TC Palaya',
       'Suraksha Nagar', 'Iblur Village', 'Yelachenahalli',
       'Basava Nagar', '2nd Block Hrbr Layout', 'Basapura',
       'Channasandra', 'Singena Agrahara', 'Mango Garden Layout',
       'Choodasandra', 'Indira Nagar', 'Sai Gardens', 'Mahadevpura',
       'Hanumanth Nagar', 'Basaveshwara Nagar', 'Kaikondrahalli',
       'Hunasamaranahalli', 'RWF West Colony', 'Bileshivale',
       'Neeladri Nagar', 'Frazer Town', 'Jaya Nagar East', 'Iggalur',
       'Banashankari', 'Chamrajpet', 'VGP Layout', 'Vasanth nagar',
       'Kalkere', 'Siddapura', 'Maragondanahalli', 'Ramamurthy Nagar',
       'Garudachar Palya', 'Roopena Agrahara', 'Gollahalli',
       'Sonnenahalli', 'D Souza Layout', 'Nagarbhavi', 'Bommanahalli',
       'Chikkalasandra', 'Dommasandra', 'Byadarahalli', 'Judicial Layout',
       'Outer Ring Road East', 'Vinayaka Nagar', 'GB Palya',
       'Ashwathnagar', 'Kasturi Nagar', 'Belathur', 'Sirsampura',
       'Devanahalli Road', 'Ejipura', 'Green Woods Layout',
       'Craig Park Layout', 'Immadihalli', 'Muneshwara Nagar',
       'Rayasandra', 'Malleshpalya', 'Parappana Agrahara',
       'Lakshmi Layout', 'Thirumenahalli', 'KPC Layout',
       'Daadys Gaarden Layout', 'Kothannur', 'Marsur', 'Karuna Nagar',
       'Kallumantapa', 'Malimakanapura', 'Medahalli',
       'Rustam Bagh Layout', 'Garden Layout', 'T K Reddy Layout',
       'Doddanekundi', 'Venkatadri Layout', 'Bommenahalli',
       'Mahaganapathy Nagar', 'HBR Layout', 'Vittal Nagar',
       'Bhuvaneshwari Nagar', 'Prithvi Layout', 'Domlur', 'Thubarahalli',
       'Jaya Mahal layout', 'BSM Extension', 'Vijinapura', 'Byrasandra',
       'Chowdeshwari Layout', 'Sector 2 HSR Layout', 'Padmanabhanagar',
       'Badavala Nagar', '4th Block Koramangala', 'Belatur',
       'Nallurhalli', 'Kereguddadahalli', 'Laxmi Sagar Layout',
       'Bannerghatta', 'Harappanahalli', 'BTM Layout', 'Kanaka Nagar',
       'NR Colony', 'Byagadadhenahalli', 'Doddabommasandra',
       'Sarjapura - Attibele Road', 'Maruthi Layout',
       'Sree Narayana Nagar', 'Tunganagara', 'Nagavara',
       'Remco Bhel Layout', 'Chokkasandra', 'Panduranga Nagar',
       'Jakkur Plantation', '1st Block Koramangala',
       'Shree Ananth Nagar Layout', 'Hoskote', 'Sector 1 HSR Layout',
       'BTM 2nd Stage', 'Ananth Nagar', 'Sundar Ram Shetty Nagar',
       'Alfa Garden Layout', 'Hoodi Layout', 'Seegehalli',
       'Gaundanapalya', '2nd Phase JP Nagar', 'Doctors Layout',
       'Basavangudi', 'Vishwapriya Nagar', 'Sarakki Nagar', 'R.T. Nagar',
       'Sector 7 HSR Layout', 'Hennur Gardens',
       'Howthinarayanappa Garden', 'Bharathi Nagar', 'Cambridge Layout',
       'Doddakannelli', 'Cox Town', 'Pulkeshi Nagar', 'Jayanagar',
       ' Bhoganhalli', 'Pai Layout', '8th block Koramangala', 'Bidadi',
       'Amruthnagar', 'Sathya Sai Layout', 'Rajiv Gandhi Nagar', 'Anekal',
       'Bhagyalakshmi Avenue', 'Doddaballapur', 'Horamavu Agara',
       'Chinnapanahalli', 'Balaji Gardens Layout', ' Rachenahalli',
       'Akshaya Vana', 'Channasandra Layout', 'Gopalkrishna Nagar',
       'Volagerekallahalli', 'Keerthi Layout', 'Shikaripalya', 'Hagadur',
       'Soundarya Layout', 'Cunningham Road', 'Dollars Layout',
       'Nagavarapalya', 'Sultan Palaya', 'Gopal Reddy Layout',
       'Thurahalli', 'Murugeshpalya', 'Kadubeesanahalli',
       'Cleveland Town', 'Kada Agrahara', 'Bellari Road',
       'Abbaiah Reddy Layout', 'Tata Nagar', ' Devarabeesana Halli',
       'Brindavan Nagar', 'Seetharampalya', 'B Narayanapura',
       'Raghuvanahalli', 'Wilson Garden', 'Challaghatta', 'KR Garden',
       'Kathriguppe', 'Sahyadri Layout', 'Bagalur', 'P Krishnappa Layout',
       'Crimson Layout', 'Kyalasanahalli', 'Chikkakammana Halli',
       'Munivenkatppa Layout', 'Vijaya Bank Layout', 'Kumarapalli',
       'Lingarajapuram', 'Vasantha Vallabha Nagar', 'Kalhalli',
       'Kumaraswami Layout', 'Hadosiddapura', 'Kachanayakanahalli',
       'Yelenahalli', '6th block Koramangala', 'Vignana Nagar',
       'Canara Bank Colony', 'Hanumigiri', 'Benson Town',
       'Akshayanagara West', 'OLd Gurappanapalya', 'Ramanjaneyanagar',
       'B Channasandra', 'New Thippasandra', 'Gattigere', 'Kamakshipalya',
       'Dollars Colony', 'Pragathi Nagar', 'HAL 2nd Stage',
       'Poorna Pragna Layout', 'Kenchenahalli', 'Kundalahalli Colony',
       'Amblipura', 'Vimanapura', 'Dodsworth Layout', 'Devasthanagalu',
       'Venugopal Reddy Layout', 'Lake City', 'S R Layout',
       'Sadduguntepalya', 'Mallathahalli', 'Doddakammanahalli',
       'Chikkathoguru', 'Richards Town', 'BCC Layout', 'Vinayak Nagar',
       'Jaladarsini Layout', 'Brooke Bond First Cross',
       'Glass Factory Layout', 'Raghavendra Nagar', 'Bhuvaneswari Nagar',
       ' Electronic City', 'Mathikere', 'Kempegowda Nagar',
       'Nyanappana Halli', 'Nelamangala', ' Whitefield',
       'Vaishnavi Layout', 'Naganathapura', 'Venkatapura',
       '3rd Phase JP Nagar', 'Devarachikkanahalli', 'Anjanapura',
       'Reliaable Tranquil Layout', 'Mailasandra', 'Chelekare',
       'Doopanahalli', 'Kattigenahalli', 'Varthur Road', 'Bethel Nagar',
       'Dollar Scheme Colony', 'Omarbagh Layout',
       'Aishwarya Crystal Layout', '2nd Block Jayanagar',
       'Nayandanahalli', 'DUO Layout', 'ISRO Layout', 'Hennur Bande',
       'Shampura', 'Langford Gardens', 'Richmond Town',
       'Vishwanatha Nagenahalli', 'Friends Colony', 'Shettigere',
       'Rahat Bagh', 'NTI Layout', 'Gunjur Palya', 'Maithri Layout',
       'Nobo Nagar', 'Krishna Reddy Layout', 'Kuvempu Layout',
       'Kaveri Nagar', 'Devi Nagar', 'Chikkabanavar',
       'Bommasandra Industrial Area', 'Stage-4 Bommanahalli',
       'Banashankari Stage V', 'Manorayana Palya', 'Sathya Layout',
       'Shanti Nagar', 'Jayamahal', 'Veer Sandra', 'Chennappa Layout',
       'Jeevan bima nagar', 'Lavakusha Nagar', 'Vittal Mallya Road',
       'Dena Bank Colony', 'Doddabidrakallu', 'NGR Layout',
       'Jnana Ganga Nagar', 'Ittamadu', 'Chokkanahalli', 'Vikram Nagar',
       'Sarvabhouma Nagar', 'Coconut Grove Layout', 'Kadabagere',
       'Anand Nagar', 'Kammanahalli', 'Janatha Colony',
       '1st Block Jayanagar', 'Veersandra', 'Kumbena Agrahara',
       'Hiremath Layout', 'Nagadevanahalli', 'Yemlur',
       'Ramamurthy Nagar Extension', 'Himagiri Meadows',
       'Narayanappa Garden', 'Kempapura', 'Carmelaram', 'Ankappa Layout',
       'Jakkuru Layout', 'Maruthi Nagar', 'Punappa Layout',
       'Pampa Extension', 'Banaswadi', 'Sanne Amanikere',
       '2nd Stage Arekere Mico Layout', 'BCMC Layout', 'Uday Nagar',
       'Chaitanya Ananya', 'Kacharakanahalli', 'Tigalarpalya',
       ' Banaswadi', 'Sugama Layout', 'Hongasandra', 'Anugrah Layout',
       'Bandepalya', 'AMS Layout', 'RPC Layout', 'Gulimangala',
       'Addischetan Layout', 'Sadaramangala', 'Konena Agrahara',
       'Raghavendra Layout', 'Varsova Layout',
       'Raja Rajeshwari Nagar 5th Stage', '7th Block Jayanagar',
       'Bande Nallasandra', 'Upkar Layout', 'Singapura Village',
       'Canara Bank Layout', 'Queens Road', 'Hosapalya', 'Jagadish Nagar',
       'Rajasree Layout', 'Chikkadunnasandra', 'Thirumalashettyhally',
       'Vibuthipura', 'Kodigehalli', 'P&T Layout', 'Varanasi',
       'Virudhu Nagar', 'Ramanashree Enclave', 'Chikku Lakshmaiah Layout',
       'Shivaji Nagar', 'Chikkagubbi', 'Kammagondahalli',
       'Yeshwanthpur Industrial Suburb', 'Thirupalya', 'Tirumanahalli',
       'ITPL', 'NRI Layout', 'Adugodi', 'Sidedahalli', 'Shanthi Pura',
       'Anwar Layout', 'Doddanakundi Industrial Area 2', 'RBI Layout',
       'Shauhardha Layout', 'Bhoopsandra', '3rd Block Koramangala',
       'Kuvempu Nagar', 'Sri Balaji Krupa Layout', 'St. John's Road',
       'Govindapura', 'Ayappa Nagar', 'Hessarghatta', 'Anantapuram',
       'Chikka Banaswadi', 'Akshayanagara East', 'Guddadahalli',
       'Kashi Nagar', 'Nanjappa Garden', 'Allalasandra',
       'Maheswari Nagar', 'Udayagiri', 'Mahalakshmi Layout',
       'Rahmath Nagar', 'Halanayakanahalli', 'Air View Colony',
       'Amam Enclave Layout', 'Anantapura', 'Mallappa Layout',
       'Devarabeesana Halli', 'Chikbasavanapura', 'Billamaranahalli',
       'Jogupalya', 'Gulakamale', 'Mahalakshmi Puram', 'Jakkasandra',
       'Cooke Town', 'Virupakshapura', 'Kenchenhalli', 'Church Street',
       'Sarvobhogam Nagar', 'Near International Airport', 'Palace Road',
       'Abshot Layout'], dtype=object)
```

In [318]:
```python
# TO CHECK THE LENGTH
len(data_clean.location.unique())
```

Out[318]:
```
634
```

In [320]:
```python
# AS THIS DATA IS RELATED TO DIMENSIONS WE DO DIMENSIONALITY REDUCTION TO GET BETTER PREDICTION
# TO REDUCE THE DIMENSIONS ONE OF THE TECHNIQUE IS OTHER CATEGORY

# SOMETIMES IF THERE ARE ENY EXTRA SPACES IN THE STRING WE HAVE TO REMOVE
# SIMPLY USE LAMBDA FUNCTION

data_clean['location'] = data_clean['location'].apply(lambda x : x.strip())
```

In [326]:
```python
# TO SEE THE THE DETAILS OF THE LOCATIONS WE CAN DEFINE LIKE LOCATIONS STATS WHICH HELPS TO SEE COUN
location_stats = data_clean.groupby('location')['location'].agg('count').sort_values(ascending = Fal
location_stats
```

Out[326]:
```
location
Whitefield                        398
Sarjapur  Road                    310
Electronic City                   238
Kanakpura Road                    216
Thanisandra                       215
                                 ... 
Kattigenahalli                      1
Kashi Nagar                         1
Kamakshipalya                       1
Kalhalli                            1
Yeshwanthpur Industrial Suburb      1
Name: location, Length: 626, dtype: int64
```

In [330]:
```python
# AS WE CAN SEE THERE ARE MANY LOCATIONS WITH ONE DATA POINT OR ONE ROW
# WE CAN USE [OTHER] METHOD --> WE CAN ASSIGN AS OTHER TO ALL LOCATION HAVING ONLY ONE POINT

len(location_stats[location_stats < 10])
```

Out[330]:
```
472
```

In [334]:
```python
# TO SEE ALL THE LOCATIONS LET'S CALL
locations_less_than_10 = location_stats[location_stats<10]
locations_less_than_10
```

Out[334]:
```
location
Sompura                           9
Devarachikkanahalli               9
R.T. Nagar                        9
Kalkere                           9
Basapura                          9
                                 ..
Kattigenahalli                    1
Kashi Nagar                       1
Kamakshipalya                     1
Kalhalli                          1
Yeshwanthpur Industrial Suburb    1
Name: location, Length: 472, dtype: int64
```

In [336]:
```python
# OVERALL ASSIGN -----> OTHER TO ALL THE LOCATIONS WITH LESS THAN 10

data_clean['location'] = data_clean['location'].apply(lambda x : 'other' if x in locations_less_than
len(data_clean['location'].unique())
```

Out[336]:
```
155
```

VISUALIZATIONS

OUTLIER DETECTION

- BOX PLOTS OR SCATTER PLOT
- IQR METHOD
- Z-SCORE

In [340]:
```python
# REMOVAL OF OUTLIERS PLAYS A CRUCIAL ROLE
# HOW LET'S CHECK OUT FOR OUTLIERS
```

In [342]:
```python
data_clean.head()
```

Out[342]:

	location	size	total_sqft	bath	price	bedrooms	price_per_sqft
0	Electronic City Phase II	2 BHK	1056.0	2.0	39.07	2	3699.810606
1	Chikka Tirupathi	4 Bedroom	2600.0	5.0	120.00	4	4615.384615
3	Lingadheeranahalli	3 BHK	1521.0	3.0	95.00	3	6245.890861
5	Whitefield	2 BHK	1170.0	2.0	38.00	2	3247.863248
11	Whitefield	4 Bedroom	2785.0	5.0	295.00	4	10592.459605

In [346]:
```python
# WE CAN ALSO CHECK FOR NUMBER OF BEDROOMS AND SQUARE_FEET
# SOMETIMES LESS SQUARE_FEET MAY HAVE MANY BEDROOMS WHICH IS UNUSUAL FOR THE DATA

data_clean[data_clean['total_sqft']/data_clean['bedrooms'] < 300].head()

# HERE WE CAN SEE THE DATA HAS ONLY 11SQFT BUT HAS 3 BEDROOM WHICH MAY LEAD TO OUTLIERS
```

Out[346]:

	location	size	total_sqft	bath	price	bedrooms	price_per_sqft
349	other	3 Bedroom	11.0	3.0	74.0	3	672727.272727
480	other	4 BHK	1150.0	4.0	260.0	4	22608.695652
483	other	5 Bedroom	1200.0	5.0	190.0	5	15833.333333
1239	Electronic City	2 BHK	550.0	1.0	16.0	2	2909.090909
2040	Electronic City	2 BHK	550.0	1.0	15.0	2	2727.272727

In [348]:
```python
data_clean.shape
```

Out[348]:
```
(7496, 7)
```

In [352]:
```python
# FOR SUCH KIND OF SMALL DATA REMOVALS WE CAN DIRECTLY CREATE NEW DATAFRAME AND REMOVE some outliers

outliers = data_clean[~(data_clean['total_sqft']/data_clean['bedrooms'] < 300)]
outliers.shape
```

Out[352]:
```
(7454, 7)
```

In [354]:
```python
outliers.price_per_sqft.describe()
```

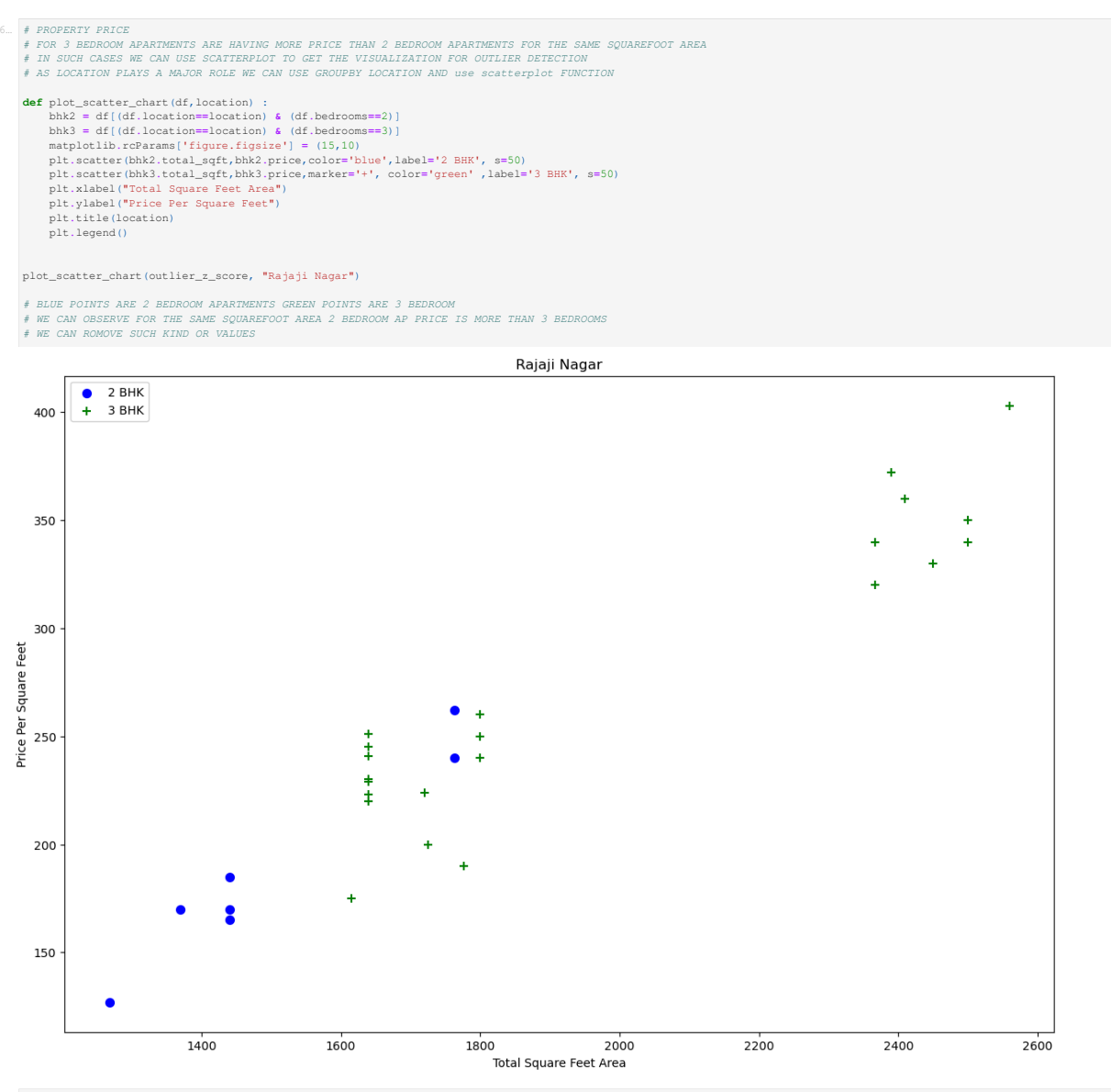
Out[354]:
```
count     7439.000000
mean      5851.020195
std       2705.465697
min        371.428571
25%       4283.397683
50%       5319.284802
75%       6693.881632
max      76530.612245
Name: price_per_sqft, dtype: float64
```

In [376]:
```python
# AS WE HAVE CREATED NEW COLUMN LET'S SEE SOME BASIC STATISTICS ON THE COLUMN ADDED
# WE CAN CLEARLY OBSERVE BY LOOKING AT MIN AND MAX VALUES WHERE VALUE 371 PER SQFT IS TOO LOW
# AND THE MAX VALUE 76530 PER SQFT IS TOO HIGH ,IT MAY BE IN ONLY RARE CASES
# NOW LET'S REMOVE PRICE PER SQUARE FEET PER LOCATION AS SOME LOCATION WILL HAVE HIGHER PRICE
# SOME LOCATIONS WILL HAVE LESS PRICE
# FOR DATAPOINTS WE HAVE TO FIND MEAN AND STANDARD DEVIATION
# FILTER OUT THE DATAPOINTS WHICH ARE BEYOND ONE SD
# Z_SCORE METHOD

def remove_pps_outliers(df):
    df_out = pd.DataFrame()
    for key, subdf in df.groupby('location'):
        mean = np.mean (subdf.price_per_sqft)
        sd = np.std(subdf.price_per_sqft )
        reduced_df = subdf[(subdf.price_per_sqft>(mean-sd)) & (subdf.price_per_sqft<=(mean+sd))]
        df_out = pd.concat([df_out, reduced_df],ignore_index=True)
    return df_out


outlier_z_score = remove_pps_outliers(outliers)
outlier_z_score.shape
```

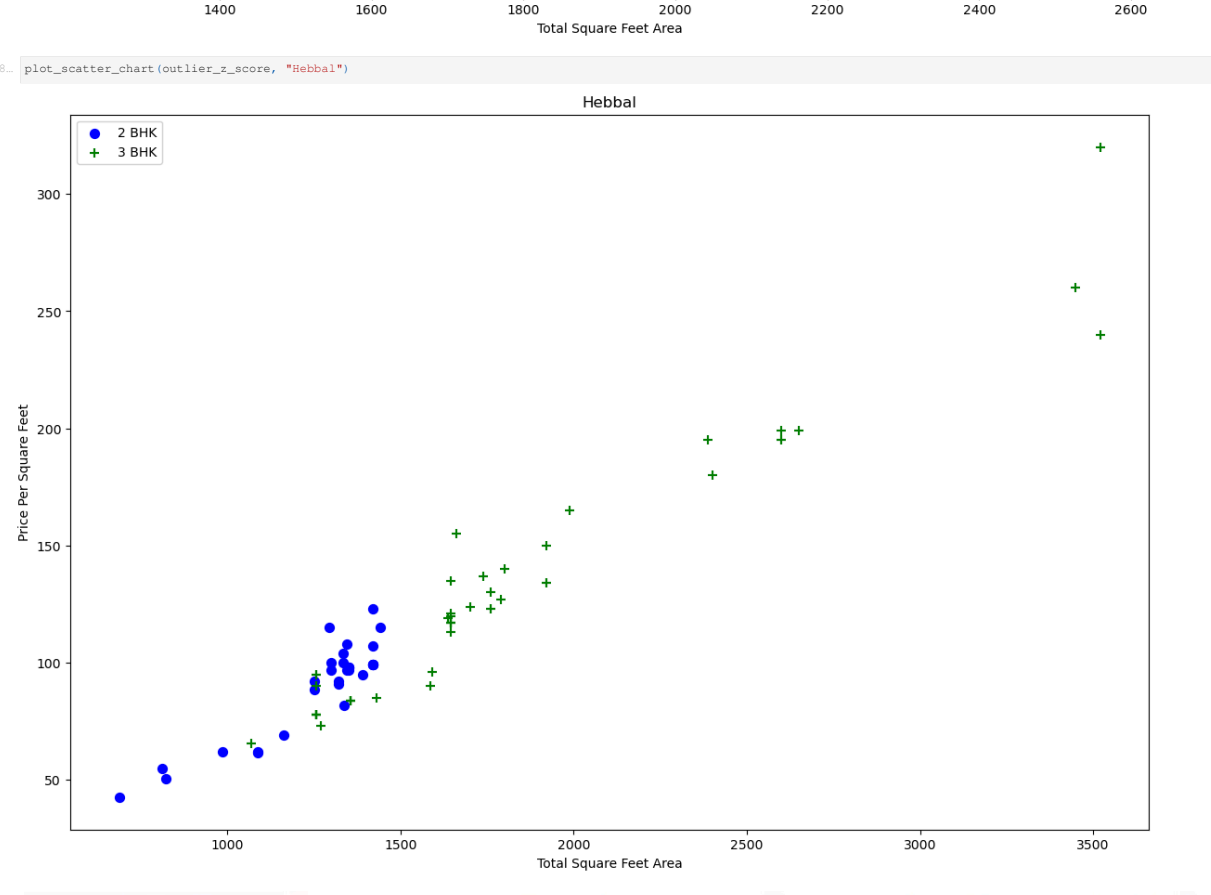
Out[376]:
```
(5844, 7)
```

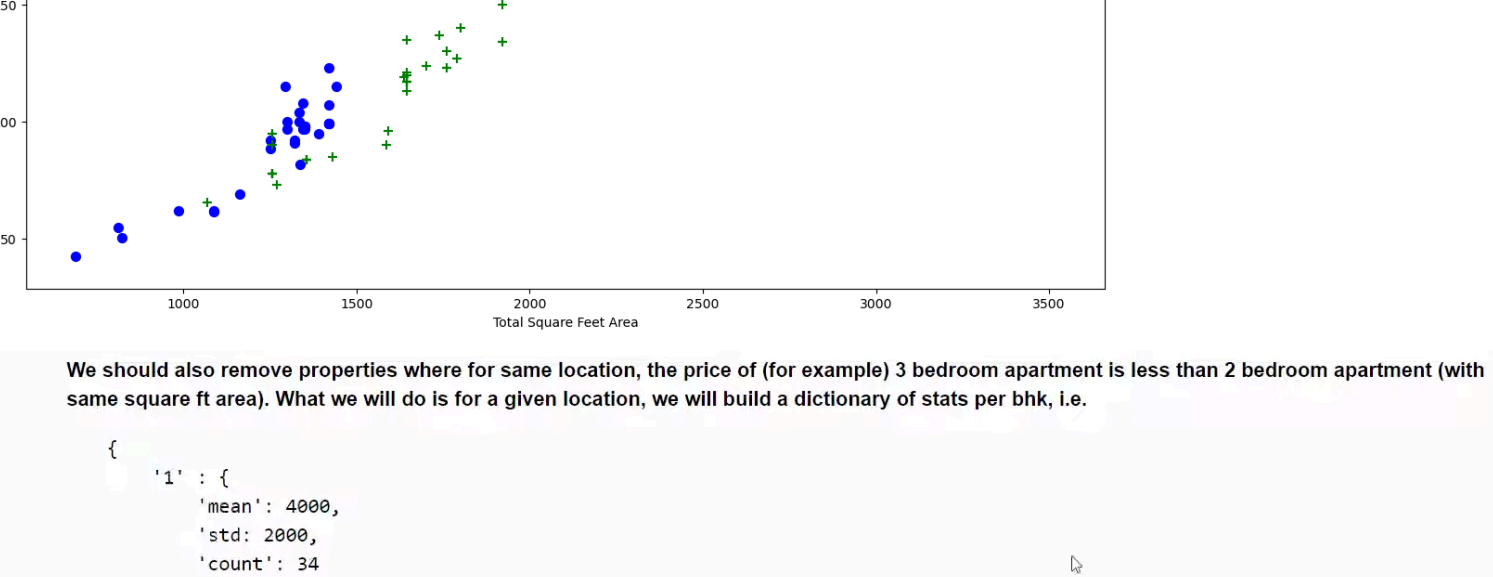
SCATTERPLOT

In [386]:
```python
# PROPERTY PRICE
# FOR 3 BEDROOM APARTMENTS ARE HAVING MORE PRICE THAN 2 BEDROOM APARTMENTS FOR THE SAME SQUAREFOOT A
# IN SUCH CASES WE CAN USE SCATTERPLOT TO GET THE VISUALIZATION FOR OUTLIER DETECTION
# AS LOCATION PLAYS A MAJOR ROLE WE CAN USE GROUPBY LOCATION AND use scatterplot FUNCTION

def plot_scatter_chart(df,location) :
    bhk2 = df[(df.location==location) & (df.bedrooms==2)]
    bhk3 = df[(df.location==location) & (df.bedrooms==3)]
    matplotlib.rcParams['figure.figsize'] = (15,10)
    plt.scatter(bhk2.total_sqft,bhk2.price,color='blue',label='2 BHK', s=50)
    plt.scatter(bhk3.total_sqft,bhk3.price,marker='+', color='green' ,label='3 BHK', s=50)
    plt.xlabel("Total Square Feet Area")
    plt.ylabel("Price Per Square Feet")
    plt.title(location)
    plt.legend()


plot_scatter_chart(outlier_z_score, "Rajaji Nagar")

# BLUE POINTS ARE 2 BEDROOM APARTMENTS GREEN POINTS ARE 3 BEDROOM
# WE CAN OBSERVE FOR THE SAME SQUAREFOOT AREA 2 BEDROOM AP PRICE IS MORE THAN 3 BEDROOMS
# WE CAN REMOVE SUCH KIND OR VALUES
```



In [388]:
```python
plot_scatter_chart(outlier_z_score, "Hebbal")
```





We should also remove properties where for same location, the price of (for example) 3 bedroom apartment is less than 2 bedroom apartment (with same square ft area). What we will do is for a given location, we will build a dictionary of stats per bhk, i.e.

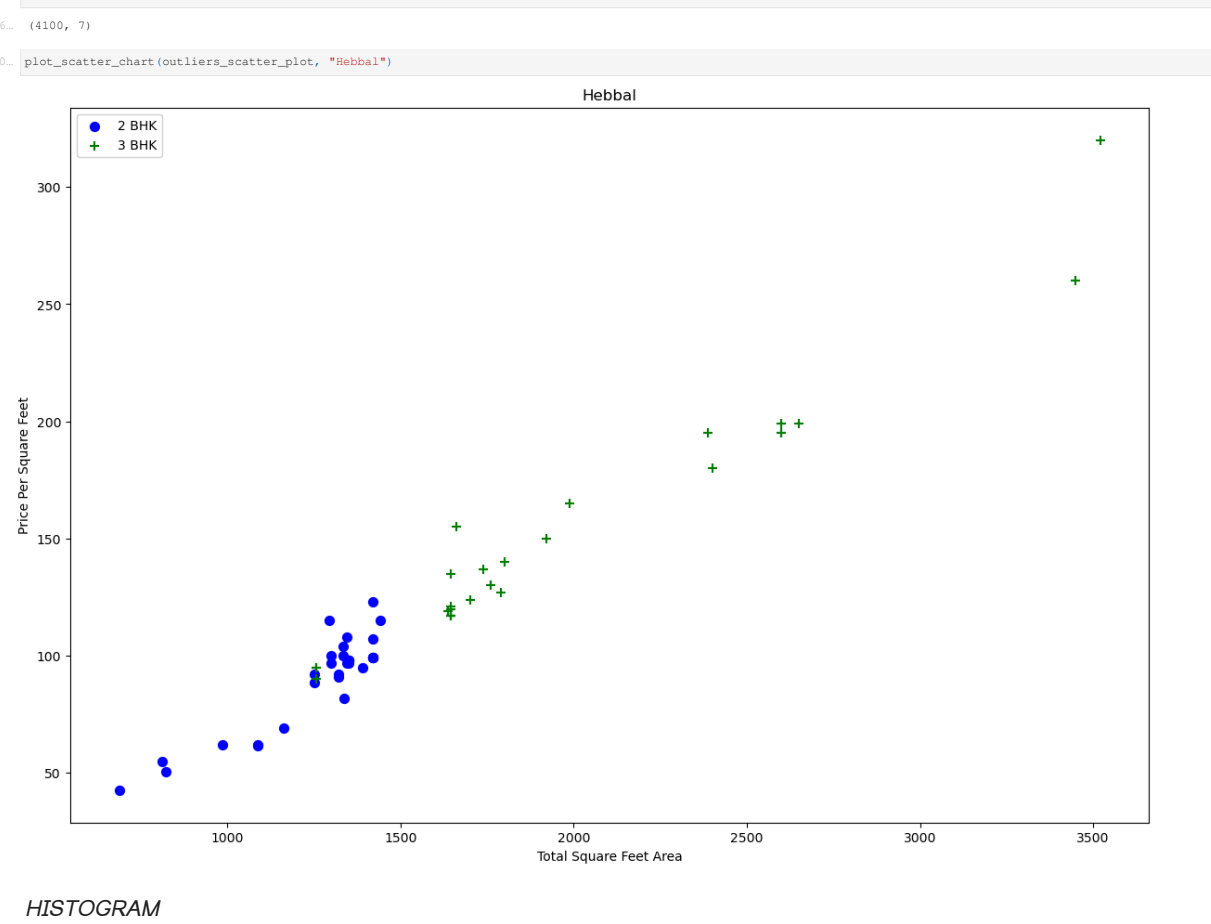
```
{
    '1' : {
        'mean': 4000,
        'std: 2000,
        'count': 34
    },
    '2' : {
        'mean': 4300,
        'std: 2300,
        'count': 22
    },
}
```

Now we can remove those 2 BHK apartments whose price_per_sqft is less than mean price_per_sqft of 1 BHK apartment

In [396]:
```python
def remove_bhk_outliers(df) :
    exclude_indices = np.array([])
    for location, location_df in df.groupby('location'):
        bhk_stats = {}
        for bhk, bhk_df in location_df.groupby('bedrooms'):
            bhk_stats[bhk] = {
                'mean' : np.mean(bhk_df.price_per_sqft),
                'std': np.std(bhk_df.price_per_sqft),
                'count': bhk_df. shape [0]
            }
        for bhk, bhk_df in location_df.groupby('bedrooms'):
            stats = bhk_stats.get(bhk-1)
            if stats and stats ['count']>5:
                exclude_indices = np. append(exclude_indices, bhk_df[bhk_df. price_per_sqft<(stats[
    return df.drop(exclude_indices, axis='index')


outliers_scatter_plot = remove_bhk_outliers(outlier_z_score)
outliers_scatter_plot.shape
```

Out[396]:
```
(4100, 7)
```

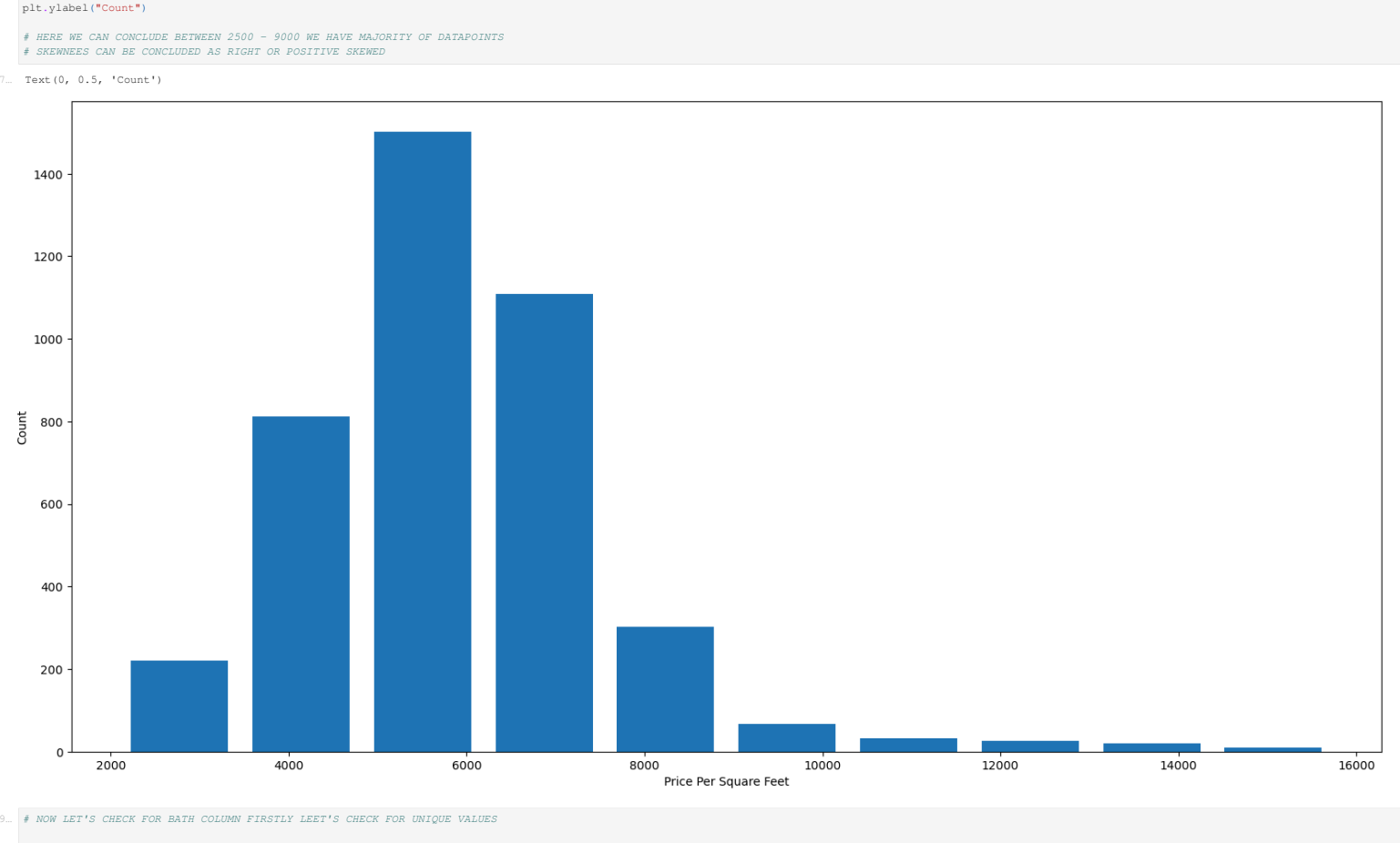
In [400]:
```python
plot_scatter_chart(outliers_scatter_plot, "Hebbal")
```



HISTOGRAM

In [407]:
```python
# NOW LET'S PLOT HISTOGRAM TO SEE HOW MANY APARTMENTS ARE THERE IN PER SQUAREFOOT AREA

import matplotlib
matplotlib.rcParams["figure.figsize"] = (20,10)
plt.hist(outliers_scatter_plot.price_per_sqft,rwidth=0.8)
plt.xlabel("Price Per Square Feet")
plt.ylabel("Count")

# HERE WE CAN CONCLUDE BETWEEN 2500 - 8000 WE HAVE MAJORITY OF DATAPOINTS
# SKEWNESS CAN BE CONCLUDED AS RIGHT OR POSITIVE SKEWED
```

Out[407]:
```
Text(0, 0.5, 'Count')
```



In [409]:
```python
# NOW LET'S CHECK FOR BATH COLUMN FIRSTLY LEET'S CHECK FOR UNIQUE VALUES

outliers_scatter_plot.bath.unique()
```

Out[409]:
```
array([4., 3., 2., 5., 1., 8., 7., 6., 9.])
```

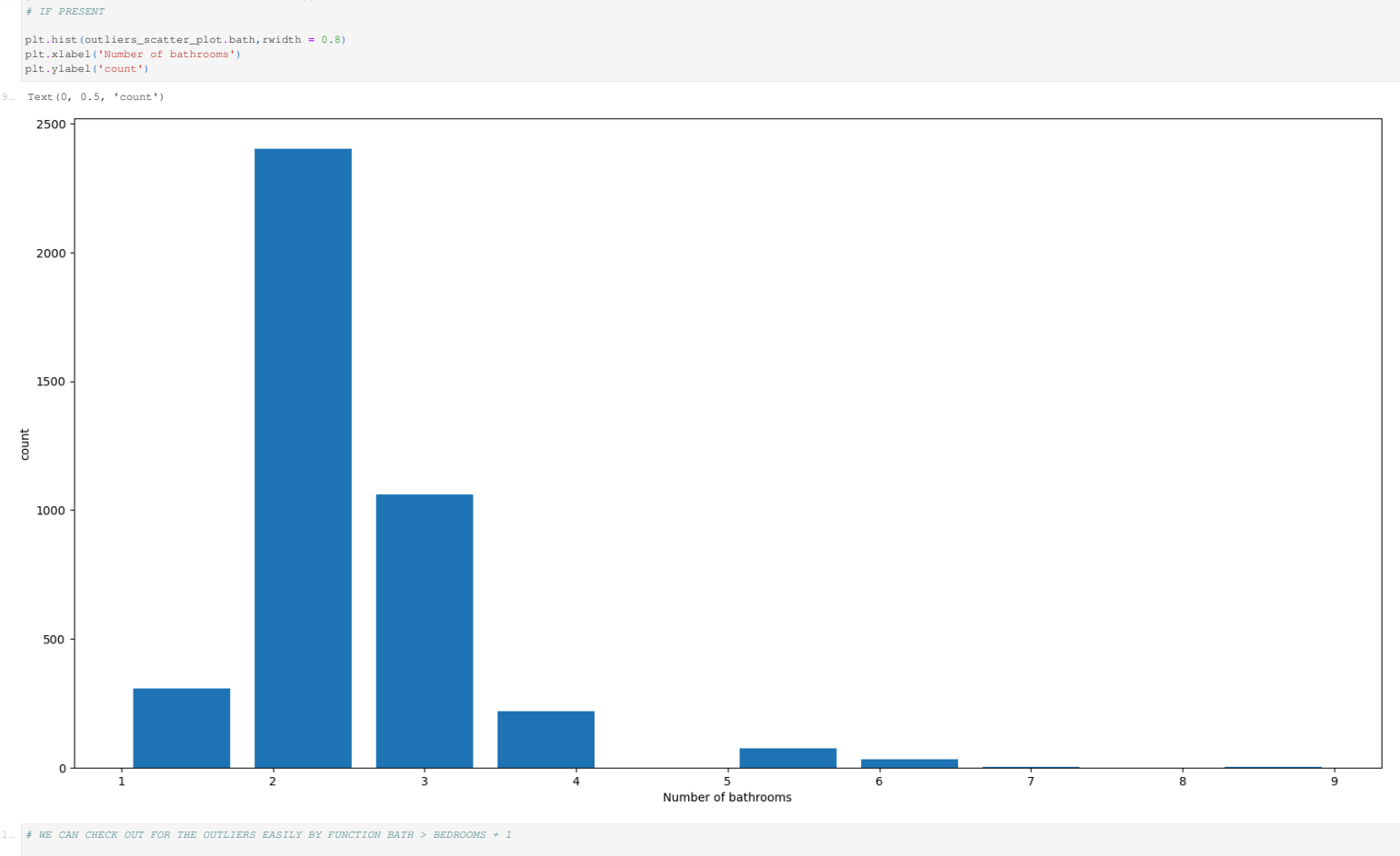
In [413]:
```python
outliers_scatter_plot[outliers_scatter_plot.bath>10]
```

Out[413]:

location	size	total_sqft	bath	price	bedrooms	price_per_sqft

In [415]:
```python
# IT WE HAVE MORE NUMBER OF BATH THAN BEDROOMS THEN WE HAVE TO REMOVE
# IT PRESENT

plt.hist(outliers_scatter_plot.bath,rwidth = 0.8)
plt.xlabel("Number of bathrooms")
plt.ylabel("count")
```

Out[415]:
```
Text(0, 0.5, 'count')
```



In [419]:
```python
# WE CAN CHECK OUT THE OUTLIERS EASILY BY POSITION BATH > BEDROOMS + 2

outliers_scatter_plot[outliers_scatter_plot.bath > outliers_scatter_plot.bedrooms + 2]
```

Out[419]:

	location	size	total_sqft	bath	price	bedrooms	price_per_sqft
3864	Thanisandra	3 BHK	1806.0	6.0	116.0	3	6423.034330
4924	other	6 BHK	11338.0	9.0	1000.0	6	8819.897689

In [423]:
```python
# WE CAN REMOVE THE OUTLIERS EASILY BY WRITING PYTHON CODE

data_model = outliers_scatter_plot[outliers_scatter_plot.bath < outliers_scatter_plot.bedrooms + 2]
data_model.shape
```

Out[423]:
```
(4053, 7)
```

In [425]:
```python
# AS WE HAVE DONE AS OUR DATA IS READY FOR MODEL BUILDING
# WE CAN DROP UNNECESSARY FEATURES FROM CLEANED DATASET
# WE CAN DROP COMMAND --> HERE WE HAVE SIZE AS BEDROOMS ,WE CAN REMOVE SIZE
# AND PRICE_PER_SQFT IS JUST ADDED FOR THE OUTLIER DETECTION

data_training = data_model.drop(['size','price_per_sqft'],axis = 'columns')
data_training.head()
```

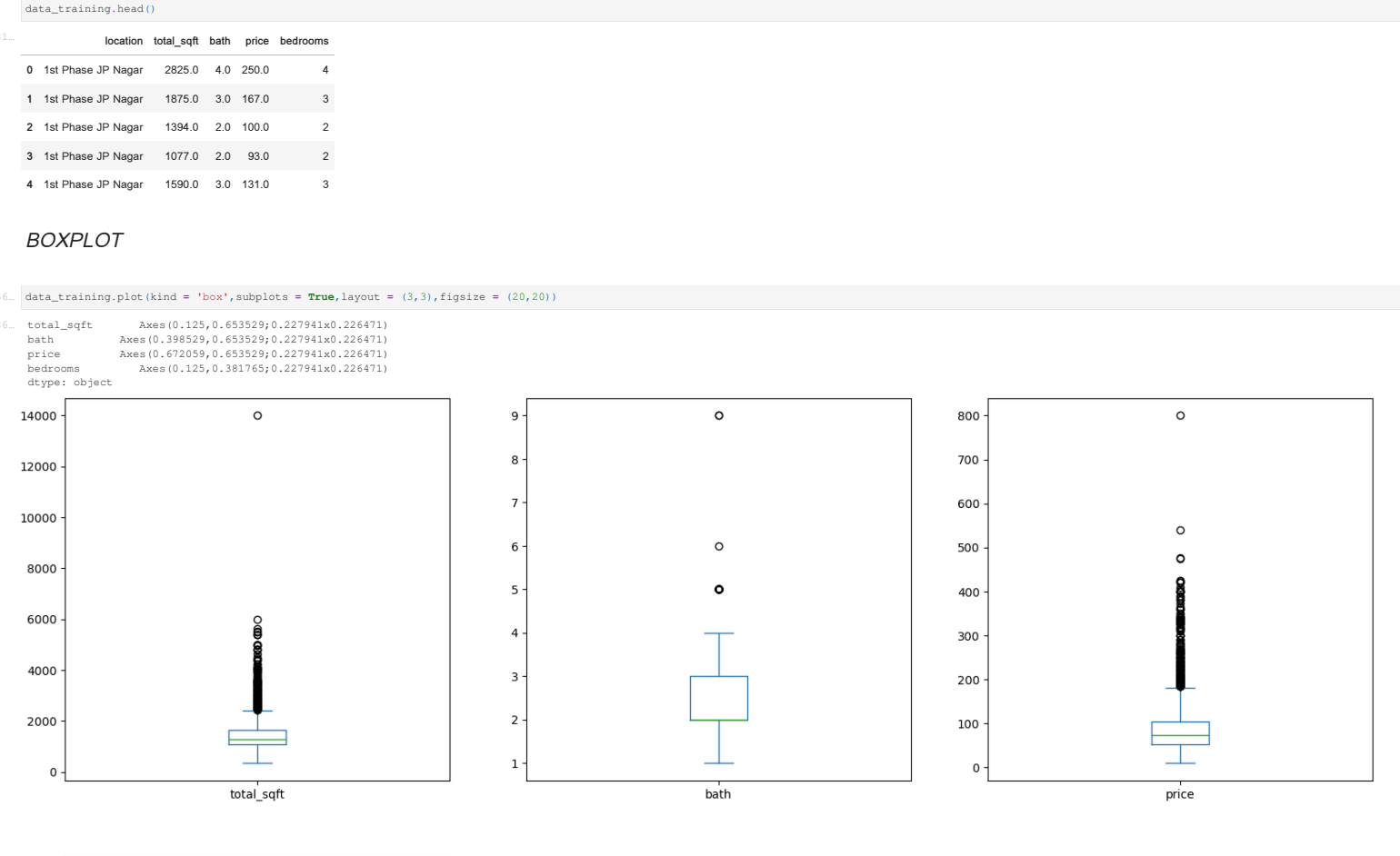
Out[425]:

	location	total_sqft	bath	price	bedrooms
0	1st Phase JP Nagar	2825.0	4.0	250.0	4
1	1st Phase JP Nagar	1875.0	3.0	167.0	3
2	1st Phase JP Nagar	1394.0	2.0	100.0	2
3	1st Phase JP Nagar	1077.0	2.0	93.0	2
4	1st Phase JP Nagar	1590.0	3.0	131.0	3

BOXPLOT

In [431]:
```python
data_training.plot(kind = 'box',subplots = True,layout = (3,3),figsize = (15,15))
```

Out[431]:
```
total_sqft        Axes(0.125,0.653529;0.227941x0.226471)
bath           Axes(0.398529,0.653529;0.227941x0.226471)
price          Axes(0.672059,0.653529;0.227941x0.226471)
bedrooms          Axes(0.125,0.381765;0.227941x0.226471)
dtype: object
```



ONE HOT ENCODING

- AS WE CAN SEE THE LOCATION IS A CATEGORICAL COLUMN
- THE ONLY WAY TO CONVERT CATEGORICAL COLUMN TO NUMERICAL IS USING ONE HOT ENCODING
- ALSO CALLED -------> DUMMIES

In [436]:
```python
# FOR EACH OF THE LOCATIONS NEW COLUMNS WILL BE CREATED
# IF THE LOCATION IS MATCHED FOR A PARTICULAR ROW
# IETS VALUE IS ONE AND ALL OTHERS ROWS AS ZERO

dummies = pd.get_dummies(data_training['location'],dtype = 'int8')
dummies
```

Out[436]:

	1st Phase JP Nagar	5th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	9th Phase JP Nagar	AECS Layout	Abbigere	Akshaya Nagar	Ambalipura	Ambedkar Nagar	...	Tumkur Road	Uttarahalli	Varthur	Vidyaranyapura	Vijayanagar	Vittasandra	Whitefield	Yelahanka	Yelahanka New Town	Yeshwanthpur	other
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0
4	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
5835	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	1
5836	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	1
5838	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	1
5840	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	1
5841	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0	1

4053 rows × 155 columns

In [438]:
```python
# NOW AS WE HAVE TO CONCAT ALL COLUMNS WITH THE DUMMY COLUMNS OF THE LOCATION
# BY USING CONCAT FUNCTION WE CAN MERGE
# NOTE :- AS WE HAVE LEARNED IN ONE HOT ENCODING TUTORIAL ,TO AVOID WITH DUMMY VARIABLE TRAP
# IT'S BETTER TO --> HAVE ONE LESS DUMMIES COLUMN
# WE CAN DIRECTLY DROP THE LAST COLUMN THAT IS OTHER

data_merged = pd.concat([data_training,dummies.drop('other',axis = 'columns')],axis = 'columns')
data_merged
```

Out[438]:

	location	total_sqft	bath	price	bedrooms	1st Phase JP Nagar	5th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	9th Phase JP Nagar	...	Tumkur Road	Uttarahalli	Varthur	Vidyaranyapura	Vijayanagar	Vittasandra	Whitefield	Yelahanka	Yelahanka New Town	Yeshwanthpur
0	1st Phase JP Nagar	2825.0	4.0	250.0	4	1	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	1st Phase JP Nagar	1875.0	3.0	167.0	3	1	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	1st Phase JP Nagar	1394.0	2.0	100.0	2	1	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	1st Phase JP Nagar	1077.0	2.0	93.0	2	1	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	1st Phase JP Nagar	1590.0	3.0	131.0	3	1	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
5835	other	1155.0	2.0	64.0	2	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
5836	other	1508.0	3.0	77.0	3	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
5838	other	1090.0	2.0	57.0	2	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
5840	other	1256.0	2.0	65.0	2	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
5841	other	1353.0	2.0	110.0	2	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

4053 rows × 159 columns

In [444]:
```python
# NOW AS WE HAVE ALREADY COVERED THE LOCATION COLUMNS SEPARATELY WE CAN EASILY DROP THE LOCATION COLUM

d_cleaned = data_merged.drop(['location'],axis = 'columns')
d_cleaned.head()
```

Out[444]:

	total_sqft	bath	price	bedrooms	1st Phase JP Nagar	5th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	9th Phase JP Nagar	AECS Layout	...	Tumkur Road	Uttarahalli	Varthur	Vidyaranyapura	Vijayanagar	Vittasandra	Whitefield	Yelahanka	Yelahanka New Town	Yeshwanthpur
0	2825.0	4.0	250.0	4	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	1875.0	3.0	167.0	3	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	1394.0	2.0	100.0	2	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	1077.0	2.0	93.0	2	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	1590.0	3.0	131.0	3	1	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

5 rows × 158 columns

CHOOSING OF [INPUT FEATURES AND TARGET VARIABLE]

- COVARIANCE
- CORELATION

In [525]:
```python
# AS WE HAVE TO PREDICT THE PRICE WE DROP THE UNWANTED OR UNRELATED COLUMNS WHILE PREDICTING PRICE
# COLUMNS WHICH DO NOT PLAY A CRUCIAL ROLE WILL BE DROPPED AS WE HAVE TO SELECT OUR X AND Y VALUE
# TO DO THIS WE CAN DO CORELATION AND COVARIANCE TO CHECK THE RELATIONSHIP AND STRENGTH BETWEEN INP

np.cov(d_cleaned['bath'],d_cleaned['price'])
```

Out[525]:
```
array([[4.02842179e-01, 3.22155021e+01],
       [3.22155021e+01, 3.35035597e+03]])
```

In [433]:
```python
np.cov(d_cleaned['total_sqft'],d_cleaned['price'])
```

Out[433]:
```
array([[383072.43342275,  32044.98776036],
       [ 32044.98776036,   3350.35596797]])
```

In [437]:
```python
np.cov(d_cleaned['bedrooms'],d_cleaned['price'])
```

Out[437]:
```
array([[1.17081337e-01, 2.77227307e+01],
       [2.77227307e+01, 3.35035597e+03]])
```

corelation

In [470]:
```python
d_cleaned[['total_sqft','price']].corr()
```

Out[470]:

	total_sqft	price
total_sqft	1.000000	0.894487
price	0.894487	1.000000

In [472]:
```python
d_cleaned[['bath','price']].corr()
```

Out[472]:

	bath	price
bath	1.000000	0.716834
price	0.716834	1.000000

In [474]:
```python
d_cleaned[['bedrooms','price']].corr()
```

Out[474]:

	bedrooms	price
bedrooms	1.000000	0.665735
price	0.665735	1.000000

In [478]:
```python
# BY THIS WE CAN CONCLUDE THAT DATA IS HAVING STRONG POSITIVE LINEAR RELATIONSHIP
# BETWEEN TOTAL_SQFT AND PRICE
# EVEN OTHER PAIRS HAVE POSITIVE LINEAR RELATIONSHIP
```

In [480]:
```python
# WHEN WE USE SCATTERPLOT FOR VISUALIZATION WE HAVE TO CHECK WHETHER THE DATA IS LINEAR OR MONOTONIC
# AS HERE THE DATA IS LINEAR SPY CIVES THE GOOD SCORE FOR RELATIONSHIP BETWEEN X AND Y

d_cleaned[['total_sqft','price']].corr(method = 'spearman')
```

Out[480]:

	total_sqft	price
total_sqft	1.000000	0.890259
price	0.890259	1.000000

In [482]:
```python
d_cleaned[['bath','price']].corr(method = 'spearman')
```

Out[482]:

	bath	price
bath	1.000000	0.752393
price	0.752393	1.000000

In [486]:
```python
d_cleaned[['bedrooms','price']].corr(method = 'spearman')
```

Out[486]:

	bedrooms	price
bedrooms	1.000000	0.735273
price	0.735273	1.000000

- AS WE CAN SEE GOOD STRENGTH AND RELATIONSHIP BETWEEN EACH COLUMN AND TARGET
- WE CAN SELECT THE MOST IMPORTANT FEATURES FOR INPUT

TRAIN - TEST SPLIT

In [488]:
```python
from sklearn.model_selection import train_test_split
```

In [494]:
```python
# AS WE KNOW WE ARE PREDICTING THE PRICES --> PRICE COLUMN ACTS AS TARGET VARIABLE
# DIRECTLY DROP THE PRICE COLUMN FROM THE DATASET AND ASSIGN ALL THE REMAINING FEATURES AS INPUT FEA

X_train = d_cleaned.drop(['price'],axis = 'columns')
X_train.head()
```

Out[494]:

	total_sqft	bath	bedrooms	1st Phase JP Nagar	5th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	9th Phase JP Nagar	...
0	2825.0	4.0	4	1	0	0	0	0	...
1	1875.0	3.0	3	1	0	0	0	0	...
2	1394.0	2.0	2	1	0	0	0	0	...
3	1077.0	2.0	2	1	0	0	0	0	...
4	1590.0	3.0	3	1	0	0	0	0	...

5 rows × 157 columns

In [520]:
```python
y_train = d_cleaned['price']
y_train.head()
```

Out[520]:
```
0    250.0
1    167.0
2    100.0
3     93.0
4    131.0
Name: price, dtype: float64
```

In [692]:
```python
# WE HAVE TO SPECIFY THE TRAIN AND TEST SIZE MANUALLY SO THAT THE MODEL FITS THE DATA!
# AS WE PASS INTO TRAIN_TEST_SPLIT WITH RANDOM STATE

X_train,X_test,y_train,y_test = train_test_split(X_train,y_train,test_size = 0.3,random_state = 10)
```

In [694]:
```python
y_test[:5]
```

Out[694]:
```
1031     64.50
71      122.00
4166     45.00
4411     92.63
2298     78.00
Name: price, dtype: float64
```

In [696]:
```python
X_test[:5]
```

Out[696]:

	total_sqft	bath	bedrooms	1st Phase JP Nagar	5th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	9th Ph...
1031	1152.0	2.0	2	0	0	0	0	0
71	1680.0	3.0	3	0	0	0	1	0
4166	825.0	1.0	1	0	0	0	0	0
4411	1639.5	3.0	3	0	0	0	0	0
2298	1314.0	2.0	2	0	0	0	0	0

5 rows × 157 columns

In [698]:
```python
X_train[:5]
```

Out[698]:

	total_sqft	bath	bedrooms	1st Phase JP Nagar	5th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	9th Ph...
77	1976.0	3.0	3	0	0	1	0	0
1545	1225.0	2.0	2	0	0	0	0	0
4059	1760.0	2.0	2	0	0	0	0	0
5539	1100.0	2.0	2	0	0	0	0	0
378	1200.0	2.0	2	0	0	0	0	0

5 rows × 157 columns

In [700]:
```python
len(X_train)
```

Out[700]:
```
2837
```

In [702]:
```python
len(y_test)
```

Out[702]:
```
1216
```

In [704]:
```python
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(2837, 157)
(2837,)
(1216, 157)
(1216,)
```

MODEL BUILDING - LINEAR REGRESSION

In [707]:
```python
from sklearn.linear_model import LinearRegression

# FIRST WE HAVE TO DEFINE THE MODEL
lr = LinearRegression()

# BY USING .FIT METHOD WE CAN TRAIN WITH ML MODEL

lr.fit(X_train,y_train)
```

Out[707]:
```
▾ LinearRegression
LinearRegression()
```

In [734]:
```python
lr.score(X_test,y_test)
```

Out[734]:
```
0.9217908144923864
```

MODEL PARAMETERS

In [710]:
```python
print(lr.intercept_)
```

```
-12.902413898650991
```

In [714]:
```python
coeff_df = pd.DataFrame(lr.coef_,X_test.columns,columns = ['coefficient'])
coeff_df
```

Out[714]:

	coefficient
total_sqft	0.072854
bath	4.788397
bedrooms	-3.447913
1st Phase JP Nagar	29.891821
5th Phase JP Nagar	-10.737055
...	...
Vittasandra	-16.431738
Whitefield	-1.282938
Yelahanka	-11.382836
Yelahanka New Town	-14.310905
Yeshwanthpur	0.612069

157 rows × 1 columns

CROSS VALIDATION

In [730]:
```python
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

cv = KFold(n_splits = 5,shuffle = True)

# TO COMPARE CROSS VAL SCORES WITH LINEAR REGRESSION
scores = cross_val_score(lr,X_train,y_train,cv = cv ,scoring = 'neg_mean_squared_error')
scores
```

Out[730]:
```
array([-2.71912784e+02, -1.50925238e+19, -2.05302655e+02, -2.20444273e+02,
       -2.80418542e+02])
```

MODEL PREDICTION

In [740]:
```python
y_pred = lr.predict(X_test)
```

In [744]:
```python
y_pred[:5]
```

Out[744]:
```
array([ 65.26359047, 119.57203629,  47.25993316, 109.28073206,
        68.07409866])
```

In [746]:
```python
# TO CHECK OUT THE PREDICTIONS
y_test[:5]
```

Out[746]:
```
1031     64.50
71      122.00
4166     45.00
4411     92.63
2298     78.00
Name: price, dtype: float64
```

In [872]:
```python
# WE CAN DEFINE A FUNCTION FOR PREDICTING THE MODEL WITH ALL THE INPUT FEATURES

def predict_price(location,total_sqft,bath,bedrooms):
    loc_index = np.where(X_train.columns == location)[0][0]

    x = np.zeros(len(X_train.columns))
    x[0] = total_sqft
    x[1] = bath
    x[2] = bedrooms
    if loc_index >= 0:
        x[loc_index] = 1

    return lr.predict([x])[0]
```

MODEL EVALUATION

In [880]:
```python
predict_price('1st Phase JP Nagar',1000,2,2)
```

Out[880]:
```
92.52467990679651
```

In [882]:
```python
predict_price('1st Phase JP Nagar',1000,3,3)
```

Out[882]:
```
93.8651637894211
```

In [884]:
```python
predict_price('Indira Nagar',1000,2,2)
```

Out[884]:
```
137.02490054688468
```

In [888]:
```python
predict_price('Indira Nagar',1000,5,5)
```

Out[888]:
```
141.04635219475844
```

In [890]:
```python
predict_price('Whitefield',1000,5,5)
```

Out[890]:
```
65.37137200724693
```

In [894]:
```python
predict_price('Yeshwanthpur',1000,2,2)
```

Out[894]:
```
63.24492742782584
```

In [910]:
```python
predict_price('Tumkur Road',1000,2,4)
```

Out[910]:
```
54.857906116147916
```

In [912]:
```python
predict_price('Indira Nagar',5000,2,4)
```

Out[912]:
```
421.54629344488177
```

In [900]:
```python
X_train.columns
```

Out[900]:
```
Index(['total_sqft', 'bath', 'bedrooms', '1st Phase JP Nagar',
       '5th Phase JP Nagar', '7th Phase JP Nagar', '8th Phase JP Nagar',
       '9th Phase JP Nagar', 'AECS Layout', 'Abbigere',
       ...
       'Tumkur Road', 'Uttarahalli', 'Varthur', 'Vidyaranyapura',
       'Vijayanagar', 'Vittasandra', 'Whitefield', 'Yelahanka',
       'Yelahanka New Town', 'Yeshwanthpur'],
      dtype='object', length=157)
```

GRID SEARCH CV()

- WE CAN USE GRID SEARCH CV TO CHECK OUT HOW DIFFERENT MODELS PERFORM ON THE DATASET

In [850]:
```python
from sklearn.model_selection import GridSearchCV
import warnings
%matplotlib inline
warnings.filterwarnings('ignore')

from sklearn.linear_model import Lasso
from sklearn.tree import DecisionTreeRegressor

# WE CAN DEFINE A FUNCTION WHICH HELPS US TO FIT ALL MODELS TOGETHER

def find_best_model_using_gridsearchcv(X,y) :
    algos = {
        'linear_regression' : {
            'model': LinearRegression(),
            'params': {
                'fit_intercept' : [True],
                'copy_X' : [True],
                'n_jobs' : [None],
                'positive': [False]
            }
        },
        'lasso': {
            'model': Lasso(),
            'params': {
                'alpha': [1,2],
                'selection': ['random', 'cyclic']
            }
        },
        'decision_tree': {
            'model': DecisionTreeRegressor(),
            'params': {
                'criterion' : ['mse', 'friedman_mse'],
                'splitter': ['best','random' ]
            }
        }
    }
    scores = []
    cv = KFold(n_splits = 5,shuffle = True)
    for algo_name ,config in algos.items():
        gs = GridSearchCV(config['model'],config['params'],cv=cv,return_train_score = False)
        gs.fit(X,y)
        scores.append({
            'model': algo_name,
            'best_score' : gs.best_score_,
            'best_params':gs.best_params_
        })
    return pd.DataFrame(scores,columns = ['model','best_score','best_params'])

find_best_model_using_gridsearchcv(X_train,y_train)
```

Out[850]:

	model	best_score	best_params
0	linear_regression	0.921059	{'copy_X': True, 'fit_intercept': True, 'n_job...
1	lasso	0.802345	{'alpha': 2, 'selection': 'random'}
2	decision_tree	0.852349	{'criterion': 'friedman_mse', 'splitter': 'best'}

In [852]:
```python
# HYPERPARAMETER TUNING IS DONE NOW
```

OVERALL LINEAR_REGRESSION GIVES THE BEST SCORE WITH ACCURACY 92%

----------- EXPORTING THE MODEL-----------------

In [916]:
```python
import pickle
with open('Bangalore_House_Prices_Prediction_model.pickle','wb') as f:
    pickle.dump(lr,f)
```

In [927]:
```python
# IMPORTANTLY AS WE NEED ALL THE COLUMNS NAMES FOR PREDICTION OF PRICES
# WE CAN ALSO IMPORT COLUMNS INTO JSON FILE BY CONVERTING ALL THE LOCATIONS INTO LOWERCASE

import json
columns = {
    'data_columns' : [col.lower() for col in X_train.columns]
}
with open('columns.json','w') as f:
    f.write(json.dumps(columns))
```

In [ ]: